



Embedded Systems

Starter Pack

The roadmap I wish someone had given me in my first year of engineering.

[@impulsetechy](#)

Electronics · Robotics · Embedded Systems

Before You Start

The Biggest Mistake Students Make

Most students learn embedded systems like this:

Arduino → ESP32 → STM32 → RTOS

Randomly. Without understanding why.

The result? Years of confusion. Shallow projects. No depth.

Instead, follow this roadmap — step by step, concept by concept.

Why This Roadmap Is Different

This isn't a list of tools. It's a progression of thinking — from understanding electrons to writing real-time firmware. Each step builds on the last. Skip one, and you'll feel it three steps later.

A personal note

I've been in electronics labs, taught embedded systems in engineering colleges, and built everything from IoT dashboards to satellite education kits. The one thing I see constantly? Students rushing to ESP32 without knowing what a transistor does. This PDF is my attempt to fix that.

Complete Embedded Systems Roadmap

Follow this order. Every step matters.

1

Electronics Fundamentals

Voltage, current, transistors, MOSFETs — the physics behind every circuit you'll ever build.

2

Embedded C

Pointers, memory, bit manipulation, structs — the language that runs the world's hardware.

3

Arduino

Your first microcontroller. Learn GPIO, ADC, PWM, timers, and interrupts with instant feedback.

4

ESP32

Wi-Fi + Bluetooth + dual-core Xtensa. Where embedded meets IoT. Industry-relevant from day one.

5

Communication Protocols

UART, I2C, SPI, CAN — every sensor and module speaks these. Learn them deeply.

6

STM32

Professional-grade ARM Cortex-M. The jump from hobbyist to engineer happens here.

7

FreeRTOS

Real-time operating system. Tasks, queues, semaphores — the foundation of production firmware.

8

Projects

Stop watching tutorials. Build something that breaks, fails, and eventually works. That's how you learn.

Embedded Engineer

You've built the foundation. Now document everything, push to GitHub, and own your narrative.

Step 1: Learn Electronics

You cannot debug what you don't understand. Before you touch a microcontroller, understand the physics.

Books

Book	Why It's Worth It
Practical Electronics for Inventors	Best beginner-to-intermediate book. Covers real circuits, not just theory. By Scherz & Monk.
The Art of Electronics	The bible. Dense but authoritative. Read after Scherz.

Websites

- All About Circuits — allaboutcircuits.com · Free textbooks + community
- Electronics Tutorials — electronics-tutorials.ws · Clean, topic-by-topic structure
- Khan Academy (Electrical Engineering) — Free, builds from zero

Goal: You Should Understand

Concept	Why It Matters in Embedded
Voltage & Current (Ohm's Law)	Every power rail, sensor reading, and GPIO signal depends on this
Transistors & MOSFETs	Motor drivers, logic level shifting, power switches — all MOSFETs
Sensors & Datasheets	Real embedded work is 80% reading datasheets
Capacitors & Decoupling	Your PCBs will randomly crash without this knowledge
Pull-up / Pull-down Resistors	Every I2C line and button circuit needs these

Step 2: Learn Embedded C

This is the step most students skip. Don't. It will come back to haunt you at every STM32 peripheral.

Recruiters rarely ask about Arduino. They ask about pointers. They ask about memory. This step alone separates your resume from 90% of other applicants.



YouTube Channels

- Jacob Sorber — best channel for C programming in embedded context
- Neso Academy — clean theory for pointers, structs, memory
- Low Level Learning — systems-level C for serious learners



Practice Platforms

- Exercism.io (C track) — hands-on problems with mentor feedback
- LeetCode (C) — not embedded-specific, but sharpens pointer skills
- Beej's Guide to C — free, comprehensive, no-fluff reference



Concepts That WILL Appear in Interviews

Topic	Real-World Embedded Use
Pointers	Memory-mapped registers, DMA buffers, callback functions
Structs & Unions	HAL peripheral config structs, sensor data packing
Bit Manipulation	Setting/clearing GPIO pins, reading status registers
Memory (Stack vs Heap)	FreeRTOS task stacks, dynamic allocation on constrained MCUs
volatile keyword	ISR flags, shared variables between tasks — critical for correctness

Step 4: Learn ESP32

The ESP32 is where embedded meets the real world. Wi-Fi, Bluetooth, dual-core, and a massive community — it's the best bridge between learning and industry.

If you have one board to start with after Arduino, it's the ESP32. Companies use it in production. Your resume needs it.

Best Resources

- Random Nerd Tutorials — randomnerdtutorials.com
The single best ESP32 tutorial site. Step-by-step projects, full code, circuit diagrams. Covers ESP-IDF, Arduino IDE, MicroPython.
- Espressif Official Docs — docs.espressif.com
Learn to read official documentation. This is what industry engineers use.
- Andreas Spiess (YouTube) — 'The Guy with the Swiss Accent'
Deep dives into ESP32, IoT, LoRa. Respected globally. Watch for engineering judgment, not just code.

Projects to Build

Level	Project Idea
Beginner	Wi-Fi Weather Station with OLED display and OpenWeatherMap API
Beginner	BLE + smartphone LED controller
Intermediate	IoT Dashboard with MQTT + Node-RED + live sensor graphs
Intermediate	Smart Home relay controller with web server
Advanced	ESP32-CAM with Telegram/WhatsApp photo alerts

Step 5: Communication Protocols

This page alone will make your PDF valuable. Communication protocols are asked in every embedded systems interview. Most students know what they are. Very few know when and why to use each.

Best Resource: All About Circuits — allaboutcircuits.com · Covers UART, SPI, I2C, CAN with waveform analysis and real implementation context.

Protocol	Wires	Speed	Topology	Best Use Case
UART	2 (TX/RX)	Up to 115200 baud	Point-to-point	Debugging, GPS, GSM modules
I2C	2 (SDA/SCL)	100k–400k Hz	Multi-device, 2 wires	Sensors, OLED displays, RTC
SPI	4 (MOSI/MISO/CLK/CS)	Up to 80 MHz	Master-slave, fast	SD cards, TFT displays, ADCs
CAN	2 (CAN H/L)	Up to 1 Mbps	Multi-master, robust	Automotive, industrial, robotics

Quick Decision Guide — Which Protocol to Use?

- **Need to debug or connect a GPS module?** Use UART
- **Multiple sensors, low pin count?** Use I2C
- **High speed display or SD card?** Use SPI
- **Noisy industrial environment, multi-node bus?** Use CAN

Step 6: Learn STM32

The jump from ESP32 to STM32 is the jump from maker to engineer. This is where serious embedded work begins.

*"If you can read STM32 documentation comfortably, you are already ahead of most students."
Real embedded engineers read datasheets — not YouTube thumbnails.*

Websites

1. Controllerstech — controllerstech.com

Probably the most recommended STM32 tutorial site among students worldwide. Step-by-step tutorials covering ADC, UART, SPI, I2C, DMA, Timers, FreeRTOS, CAN, and more. The best place to start.

Best for:  Beginners → Intermediate

2. STM32 Official Documentation

Every company expects their engineers to read documentation. Not watch tutorials — read official reference manuals. This is non-negotiable if you want to work professionally.

Best for:  Intermediate  Job Preparation

3. Embedded Artistry — embeddedartistry.com

Professional embedded concepts beyond STM32 — memory management, firmware architecture, RTOS patterns, and hardware bring-up practices used in industry.

Best for:  Beyond STM32  Serious Engineers

YouTube Channels

Phil's Lab — For Serious Students

Professional embedded engineer. Covers STM32, custom PCB design, RTOS, and real firmware architecture. Industry-level explanations — not dumbed down.

Controllerstech YouTube — For Beginners

Easy practical tutorials. STM32CubeIDE walkthroughs. Perfect for first-time STM32 learners who need to see it working before understanding why.

Community

- STM32 Community Forum — st.com/content/st_com/en/support/community

Real engineers answering real questions. Learn debugging by watching others debug. Best for:  Problem Solving

STM32 Learning Phases

Don't jump to FreeRTOS before you understand timers. Here's the correct progression:

Phase	Topic	What You'll Build
1	Embedded C + GPIO	LED blinker, button debounce
2	ADC	Potentiometer → UART → PC
3	UART	PC terminal communication
4	I2C	Temperature sensor on OLED
5	SPI	TFT display with live data
6	Timers	PWM motor control, frequency gen
7	DMA	High-speed ADC sampling
8	Interrupts	External + timer + peripheral ISRs
9	FreeRTOS	Multi-task sensor + comms system
10	Projects	Sensor node, motor controller, CAN bus
11	Portfolio	GitHub + documentation + demo video
12	Job Ready	Resume  GitHub  LinkedIn 

Step 7: Learn FreeRTOS

Most students don't know where to start with RTOS. This is the clearest path I know.

FreeRTOS is not just an "advanced topic". It's the difference between firmware that works on a desk and firmware that runs reliably in a product for 10 years. Learn it before you look for a job.



Resources

- FreeRTOS Official Documentation — freertos.org
The official source. Used in industry. Read the 'Getting Started' and 'RTOS Fundamentals' sections first.
- Shawn Hymel (Digi-Key on YouTube) — FreeRTOS Series
Best practical introduction to FreeRTOS on real hardware. Goes step-by-step with STM32 and ESP-IDF. Professional production quality.
- Phil's Lab (YouTube) — RTOS + STM32 integration
More advanced — covers real firmware architecture using RTOS. Watch after Shawn Hymel.



Core Concepts You Must Understand

Concept	What It Does in Practice
Tasks	Run independent functions concurrently. Each task has its own stack, priority, and state machine.
Queues	Safe data passing between tasks. Producer-consumer pattern for sensor data pipelines.
Semaphores	Signaling between tasks or from ISRs. Binary semaphore = "event happened".
Mutexes	Protect shared resources (UART, SPI bus) from being accessed by two tasks simultaneously.
Timers	Software timers that trigger callbacks without blocking tasks. Heartbeat patterns, timeouts.

Projects to Build

- LED task scheduler — 3 tasks at different frequencies, no delay()
- UART with queues — receive serial data in ISR, process in task
- Multi-sensor pipeline — ADC + I2C sensors feeding a display task via queues
- Motor control with watchdog — RTOS task + safety timeout

Tools Every Embedded Engineer Uses

This page is gold. Master these tools and you will feel at home in any embedded team from day one.

Tool	What It Does	Why You Need It
VS Code	Code editor with embedded extensions	Industry standard IDE — faster than CubeIDE for most work
PlatformIO	Build system + library manager	Replaces Arduino IDE for serious development. Multi-platform.
STM32CubeIDE	Official ST IDE + CubeMX	HAL code generation + debugger integration in one tool
GitHub	Version control + portfolio	Every project you build lives here. Recruiters check it.
Logic Analyzer	Protocol decoder — UART/I2C/SPI	You will not debug communication bugs without this. Buy one.
Oscilloscope	Signal visualization	PWM, I2C clock stretching, power glitches — you need to see them
Serial Monitor	UART debug terminal	Printf debugging is underrated. Master it before fancy tools.

Portfolio Projects

Stop collecting tutorials. Start collecting finished projects. One documented project beats ten half-built ones.

The formula that works: Board + Protocol + Real Use Case + GitHub + 60-second demo video = Interview conversation starter

Beginner Projects

Project	What You'll Learn
Arduino Keypad Password Lock System with Servo, LCD & Indicator Alerts	GPIO, servo motors, sensor interfacing, LCD
Arduino Bluetooth Controlled 4WD Car	PWM, motor drivers, real-time sensor response
ESP32 Temperature Logger	Wi-Fi, HTTP client, JSON parsing, REST API

Intermediate Projects

Project	What You'll Learn
IoT Dashboard with MQTT + Node-RED	Pub/sub architecture, real-time data pipelines
STM32 Sensor Node (I2C + UART)	HAL library, register-level debugging, power management
STM32 Motor Controller (PWM + PID)	Timer peripheral, closed-loop control theory

Advanced Projects

Project	What You'll Learn
FreeRTOS Multi-Task Sensor System	Task scheduling, queues, semaphores — RTOS in practice
CAN Bus Multi-Node Network	Automotive protocol, message arbitration, STM32 FDCAN peripheral
Custom PCB with STM32	KiCad/EasyEDA, schematic → layout → fab → bring-up

How To Get Your First Embedded Job

This page will get shared. Screenshot it, save it, send it to your friends. This is the advice nobody gives you until it's too late.

Don't Focus On

-  Collecting certificates with no projects
-  Watching 200 tutorials without building anything
-  Random courses with no clear direction
-  Waiting until you feel "ready"
-  Arduino forever (it's a starting point, not a destination)

Focus On

-  Projects that solve a real problem
-  GitHub with clean READMEs and schematics
-  Documenting your build process
-  Problem solving — debugging skills
-  A portfolio that tells a story



Marketing Angle

The best marketing is not selling. If your GitHub shows you built an autonomous robot, wrote clean documentation, and solved real problems — you don't need to pitch yourself. Let the work speak.

Questions You Should Be Able to Answer

Read this slowly. Answer each honestly — without Google. The gaps you find here are your next learning targets.

If you can answer all of these confidently, you are ready to apply for embedded systems roles.
If you can't answer 5 or more — you know exactly what to study next.

Embedded C & Fundamentals

- Q: What is a pointer?** — *Can you explain what `*ptr = value` does at the memory level?*
- Q: What is the volatile keyword?** — *When would you use it and why does it matter in embedded code?*
- Q: Stack vs. Heap in embedded context?** — *What happens when you run out of stack on a microcontroller?*
- Q: What is bit manipulation?** — *How do you set bit 3, clear bit 5, and toggle bit 7 of a register?*

Protocols

- Q: What is UART?** — *How does it work without a clock line? What is baud rate?*
- Q: Difference between SPI and I2C?** — *When would you choose one over the other?*
- Q: What is CAN bus and why is it used in cars?** — *What makes it robust in noisy environments?*
- Q: What is I2C addressing?** — *How do you connect two sensors with the same default address?*

RTOS & STM32

- Q: What is a semaphore?** — *Difference between binary semaphore and mutex?*
- Q: Why use RTOS?** — *What problem does it solve that a superloop can't?*
- Q: Difference between STM32 and ESP32?** — *When would you use one over the other professionally?*
- Q: What is DMA?** — *Why use DMA instead of CPU-driven data transfer?*

Bonus Resources

Everything below is free. Every link is something I have personally used or deeply trust. No filler.

Electronics

- All About Circuits — allaboutcircuits.com
- Electronics Tutorials — electronics-tutorials.ws
- Khan Academy Electrical Engineering — khanacademy.org
- Practical Electronics for Inventors (book) — Paul Scherz & Simon Monk

Embedded Systems

- Embedded Artistry — embeddedartistry.com
- Controllerstech — controllerstech.com (STM32)
- FreeRTOS Documentation — freertos.org
- Shawn Hymel / Digi-Key (YouTube) — FreeRTOS series
- Phil's Lab (YouTube) — STM32, PCB, RTOS
- Jacob Sorber (YouTube) — Embedded C and systems programming
- Low Level Learning (YouTube) — Systems-level C
- Andreas Spiess (YouTube) — IoT and ESP32 engineering
- Random Nerd Tutorials — randomnerdtutorials.com (ESP32)
- Espressif Official Docs — docs.espressif.com

Robotics

- ROS 2 Docs — docs.ros.org
- The Construct (YouTube) — ROS 2 practical tutorials
- Kevin Lynch — Modern Robotics (free textbook on Coursera)

Space Technology

- NASA OpenMCT — nasa.github.io/openmct
- LibreCube — librecube.org (open-source CubeSat platform)
- AMSAT — amsat.org (amateur satellite community)
- ESA Education — esa.int/Education

Follow for more:

IG

[@impulsetechy](#)

Linkedin

<https://www.linkedin.com/in/yogesh-bawane/>

YouTube

<https://www.youtube.com/@ImpulseTech>

I share practical engineering content every week.

Electronics · Coding · Innovation

Linkedin